

SKALA

AI 서비스를 위한 SW 기초 Full-stack Engineering

SQLD 자격증 안내

2026.7

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[별첨] SQLD 자격 취득

- 
- ① SQLD 시험 개요 및 출제 영역 소개
 - ② SQLD 핵심 이론 정리
 - ③ SQLD 기출 문제 풀이
 - ④ PostgreSQL 환경에서의 SQLD 응용 실습

KDT 데이터베이스 – DBMS 및 SQL 활용

1. SQLD 시험 개요 및 출제 영역 소개

- SQLD 자격증이란?
- 응시 자격 및 응시 절차
- 합격 기준 및 점수 체계
- 출제 영역 요약
- 대비 전략

SQLD 자격증이란?

주요 내용



정해진 시간



70점 이상

• SQL Developer (SQLD)

- SQL 개발자 자격증

주관 기관



• 한국데이터산업진흥원 (K-DATA)

- 학습한 내용을 요약, 온라인
실습, 테스트 시행(Hurdle)

목표 및 활용 분야



• 목표

- DB 기반의 데이터 설계, 질의,
최적화 능력 검정

• 활용분야

- DB 설계, 백엔드 개발,
데이터 분석, 품질관리

단순한 SQL문 작성 능력이 아닌 데이터 모델링부터 SQL의 작성,
성능 최적화까지 현업의 실무 능력을 포괄적으로 검정

*데이터 중심 서비스에서 SQL은 '공기'와 같다

응시 자격 및 응시 절차

【 응시 자격/응시 절차 】

▪ 응시 자격

- 누구나 응시 가능 (학력·경력 제한 없음)

▪ 응시 절차

- 원서 접수: K-DATA 홈페이지
- 시험 응시: 연 4회, CBT 방식 (지정 고사장에서 컴퓨터로 응시)
- 합격 발표: 시험 후 약 2주

- **SQLD**는 대학생, 현업 개발자, 데이터 엔지니어 모두 응시 가능
- 별도의 경력 요건이 없어서 첫 자격증으로 하는 사람 多

합격 기준 및 점수 체계

【 합격 기준 】

■ 기준

- 총점 100점 만점 중 70점 이상 합격
- 문항 수: 약 60~65문항
- 객관식 4지선다형 (계산/쿼리 포함)

【 채점 방식 예시 】

영역	비중	예시 점수(100점 기준)
데이터 모델링	20%	14점
SQL 기본/활용	40%	28점
응용 SQL/성능	40%	30점
합계	100%	72점 (합격)

【 출제 영역 】

영역	세부 주제
데이터 모델링 (20%)	ERD, 정규화, 엔티티/속성/관계
SQL 기본/활용 (40%)	SELECT, JOIN, GROUP BY, 서브쿼리 등
응용 SQL 및 성능 (40%)	인덱스, 뷰, 파티셔닝, 트랜잭션, 최적화 등

【 전략 】

유형	전략
전공자/비전공자	정규화/키 개념 정리 + 실습 기반 SQL 연습
실무자	성능 최적화 파트 집중 (EXPLAIN, 인덱스)
데이터 분석	SQL 기본 + 집계/서브쿼리 실습

- 이론: [데이터베이스 개론 (홍진호)] + [SQL 활용서]
- 실습: PostgreSQL / Oracle 기반 SQL 작성
- 기출분석: SQLD 기출 복원 문제 분석 (10회분)

- CBT 시험은 타이핑 없음 → SQL 결과 예측 위주로 준비
- COUNT(*), JOIN, RANK() 등 자주 나오는 문법은 암기 필수
- SQLD 기출문제 복원 사이트 활용 (ex. dataq.or.kr, 모의고사 PDF)
- Colab or pgAdmin 실습으로 빠르게 실력 향상

SQLD Pass!

KDT 데이터베이스 – DBMS 및 SQL 활용

2. SQLD 핵심 이론 정리

- 데이터 모델링 기본
- SQL 기본 및 활용
- 응용 SQL 및 성능 튜닝

데이터 모델링 기본

【 엔터티/속성/관계/식별자/비식별자 】

- **엔터티(Entity):** 현실 세계의 객체 (예: 사용자, 주문, 상품)
 - Strong Entity: 기본 키가 독립적
 - Weak Entity: 외래 키에 의존
- **속성(Attribute):** 엔터티 특징(예: 사용자명, 이메일)
 - 단순 vs 복합, 단일 vs 다중값, 저장 vs 파생
- **관계(Relationship):** 엔터티간 연결
 - 1:1, 1:N, N:M (→ 교차 엔터티 생성 필요)
- **식별자/비식별자 관계**
 - 식별자 관계: 자식 테이블의 PK가 부모의 PK를 포함 (강한 연결)
예: 주문(Order) → 주문 상세(Order_Detail)
 - 비식별자 관계: 자식의 PK는 독립, 외래키로만 연결 (유연성 확보)

【 정규화/비정규화 】

단계	내용	목적
1NF	원자값	중복 컬럼 제거
2NF	부분 함수 종속 제거	복합키 분리
3NF	이행적 종속 제거	컬럼간 독립성

항목	정규화	비정규화
장점	중복 최소화, 무결성 보장	조회 성능 향상, 조인 감소
단점	조인 증가로 성능 저하	데이터 중복, 무결성 위험
예시	OLTP, 트랜잭션 중심	OLAP, 리포팅/분석 시스템

【 SELECT 기본 구조 】

```
SELECT 컬럼명  
FROM 테이블  
WHERE 조건  
GROUP BY 그룹기준  
HAVING 그룹조건  
ORDER BY 정렬기준;
```

【 조인 문법 및 유형 】

유형	설명	예시
INNER JOIN	매칭되는 행만	고객과 주문
LEFT JOIN	왼쪽 테이블 모두	고객 + 주문 없는 고객
RIGHT JOIN	오른쪽 테이블 모두	잘 안 씀
FULL JOIN	양쪽 모두 포함	PostgreSQL에서 FULL OUTER

【 GROUP BY와 서브쿼리 】

- **GROUP BY:** 집계 함수(Aggregate) 사용 시 필수
- **HAVING:** 집계 결과에 조건
- 서브쿼리
 - 스칼라 서브쿼리: SELECT 절
 - IN 서브쿼리: WHERE 절
 - EXISTS 서브쿼리: 성능 우수 (반복 조건)

【 윈도우 함수 】

함수	설명	예시
ROW_NUMBER()	순번 부여	ROW_NUMBER() OVER(ORDER BY price DESC)
RANK()	동일값에 동일 순위	
SUM() OVER	누적 합	SUM(sales) OVER(PARTITION BY year)

【 파티셔닝 】

종류	설명	예시
Range	범위별 분할	날짜별 주문
List	목록별 분할	지역 코드
Hash	해시함수 기반	균등 분산 목적

【 윈도우 함수 】

함수	설명	예시
ROW_NUMBER()	순번 부여	ROW_NUMBER() OVER(ORDER BY price DESC)
RANK()	동일값에 동일 순위	
SUM() OVER	누적 합	SUM(sales) OVER(PARTITION BY year)

【 CTE (공통 테이블 표현식) 】

```
WITH recent_orders AS (  
  SELECT * FROM orders WHERE order_date >=  
  CURRENT_DATE - INTERVAL '30 days'  
)  
SELECT * FROM recent_orders WHERE total > 100;
```

【 VIEW 】

- 복잡한 쿼리 재사용 가능
- 가상 테이블로 활용

```
CREATE VIEW top_customers AS  
SELECT customer_id, SUM(total) AS total_spent  
FROM orders GROUP BY customer_id HAVING  
SUM(total) > 1000;
```

응용 SQL 및 성능 튜닝

【 인덱스 원리 】

- **B-Tree** 인덱스: 대부분 기본 인덱스
- **Hash** 인덱스: 등가 비교에만 사용 (**PostgreSQL** 제한적)
- **Composite** 인덱스: 다중 컬럼 (선행 조건 순서 중요)

```
CREATE TABLE orders (  
  id SERIAL, total NUMERIC, order_date DATE  
) PARTITION BY RANGE(order_date);
```

【 집계 성능 비교 】

- **GROUP BY + JOIN** vs 사전 집계된 테이블 활용
- 물리적 집계 테이블을 사용할 경우 **OLAP**에 유리

응용 SQL 및 성능 튜닝

【 WHERE vs HAVING 】

항목	WHERE	HAVING
시점	그룹화 전	그룹화 후
사용 조건	일반 컬럼	집계 함수

-- WHERE: 행 필터링

```
SELECT * FROM sales WHERE region = '서울';
```

-- HAVING: 그룹 필터링

```
SELECT region, SUM(total) FROM sales GROUP BY
region HAVING SUM(total) > 1000;
```

【 트랜잭션 처리 흐름 】

```
BEGIN;
```

```
UPDATE account SET balance = balance - 100 WHERE id = 1;
```

```
UPDATE account SET balance = balance + 100 WHERE id = 2;
```

```
COMMIT;
```

▪ ACID 원칙

- Atomicity: 원자성
- Consistency: 일관성
- Isolation: 격리성
- Durability: 지속성

▪ PostgreSQL 트랜잭션 예외 처리

```
BEGIN;
```

```
SAVEPOINT sp1;
```

```
-- 오류 발생 시
```

```
ROLLBACK TO SAVEPOINT sp1;
```

```
-- 아니면 계속 진행
```

```
COMMIT;
```

KDT 데이터베이스 – DBMS 및 SQL 활용

3. 쿼리 호환 / 실전 모의고사 / 요약노트

- PostgreSQL vs SQL Server_Oracle 쿼리 호환 매핑
- SQLD_실전모의고사_100문제
- SQLD_요약노트_PostgreSQL

쿼리 호환 매핑

【 쿼리 호환 매핑 】

DB	SQL문 비교
PostgreSQL	MS SQL, Oracle 과의 쿼리 비교

기능/설명	PostgreSQL	SQL Server	Oracle	비고
기본 LIMIT	LIMIT 10	TOP 10 또는 OFFSET FETCH	ROWNUM 또는 FETCH FIRST	행 제한 방식 차이
OFFSET	OFFSET 10	OFFSET 10 ROWS	OFFSET 10 ROWS	SQL:2008 이후 표준
문자열 연결	'a' 'b'	+		'a' + 'b'
현재 시각	NOW()	GETDATE()	SYSDATE	타입/형식 주의
AUTO_INCREMENT	SERIAL, GENERATED ALWAYS AS IDENTITY	IDENTITY(1,1)	GENERATED ...	ID 생성 방식
IF 조건	CASE WHEN ... THEN ...	동일	동일	SQL 표준
최상위 N개	LIMIT N	TOP N	ROWNUM <= N 또는 FETCH FIRST N ROWS ONLY	성능 최적화 시 중요
서브쿼리 필터	IN (SELECT ...)	동일	동일	
윈도우 함수	RANK() OVER (...)	동일	동일	완전 호환
JSON 필드 추출	jsonb ->> 'key'	JSON_VALUE()	JSON_VALUE()	PostgreSQL은 -> / ->>
동적 서브쿼리 조인	LATERAL	CROSS APPLY / OUTER APPLY	CROSS APPLY / OUTER APPLY	기능 같음, 문법만 다름
재귀 쿼리	WITH RECURSIVE ...	동일	동일	표준 SQL
UPSERT	INSERT ... ON CONFLICT DO ...	MERGE INTO	MERGE INTO	PostgreSQL만 ON CONFLICT
텍스트 배열	TEXT[]	없음 (별도 테이블)	없음	JSON 또는 정규화 필요
GIN/GiST 인덱스	지원함	미지원	미지원	PostgreSQL만 전문 검색 특화
FULL TEXT 검색	to_tsvector() + @@	CONTAINS() (FTS 설정 필요)	CONTAINS()	설정 방식 다름
RANDOM	RANDOM()	NEWID() / RAND()	DBMS_RANDOM.VALUE	난수 발생

JOIN/APPLY 관련 매핑 요약			
목적	PostgreSQL	SQL Server / Oracle	의미
왼쪽 각 행에 따라 오른쪽 쿼리 실행	LATERAL	CROSS APPLY	INNER 성격
위 + 결과 없어도 포함	LEFT JOIN LATERAL	OUTER APPLY	OUTER 성격
단순 모든 행 조합	CROSS JOIN	CROSS JOIN	디카르트곱 (JOIN 조건이 없음 - 전체 테이블 데이터를 조합으로 조회)

공통 채널에 공유한 자료 참고



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.